
SMS Spam Detection System

A Comparative Machine Learning Study & Production Deployment

Prepared By: Machine Learning Intern

Project Category: Natural Language Processing (NLP)

Deployment Stack: Scikit-Learn, Streamlit, Joblib

Date: June 21, 2026

Status: Production Ready

1. Executive Summary & Objective

SMS communication remains a prominent channel for fast notifications, transactional verification codes, and personal updates. However, it is also highly susceptible to unsolicited promotions, scams, and malicious phishing attempts, collectively categorized as **Spam**. This project builds a robust, data-driven machine learning system to classify incoming text messages as either **Ham (Legitimate)** or **Spam** in real-time. By utilizing term-weighting features and comparing probabilistic and linear classifiers, the final system selects the best performing model based on the F1-score metric and deploys it via an interactive Streamlit web application.

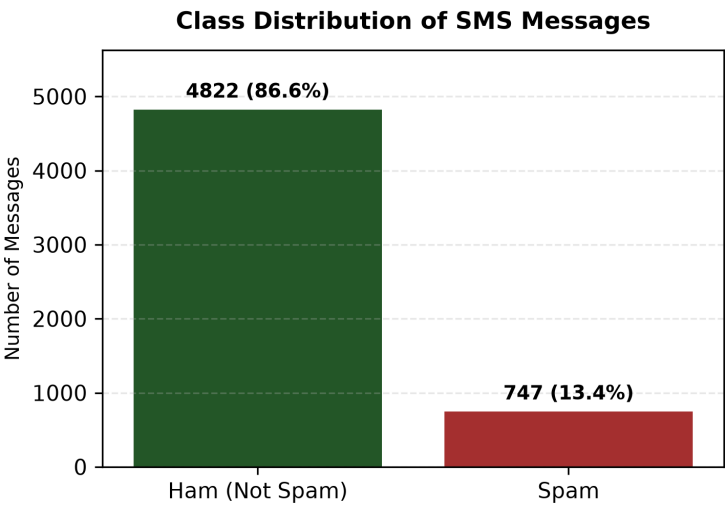
2. Dataset Analysis & Preprocessing

The model was trained and evaluated on the public **SMS Spam Collection Dataset**, which contains 5,572 raw English SMS messages annotated with ground truth labels (*ham* or *spam*).

To construct clean, numeric representations of language, text messages are subjected to a multi-stage preprocessing pipeline:

- **Lowercasing:** All text is normalized to lowercase to prevent terms like 'Win' and 'win' from being indexed as separate features.
- **Punctuation & Character Filtering:** Special symbols, punctuation, and digits are removed using regular expressions, focusing purely on alpha characters.
- **Whitespace Standardization:** Multiple consecutive spaces and leading/trailing spaces are stripped to ensure clean word delimiters.
- **Label Mapping:** Categorical targets are encoded as binary numeric labels, where **Ham = 0** and **Spam = 1**.

Figure 1: Dataset class distribution showing a significant class imbalance typical of real-world spam detection scenarios.



3. Feature Engineering & Vectorization

Machine learning models require numerical feature vectors. We utilize the **TF-IDF (Term Frequency-Inverse Document Frequency) Vectorizer**. TF-IDF measures how important a word is to a document relative to the entire corpus. The mathematical representation is:

$$\begin{aligned} \text{TF-IDF}(t, d, D) &= \text{TF}(t, d) \times \text{IDF}(t, D) \\ \text{Where: } \text{TF}(t, d) &= f_{t,d} / \sum(f_{t',d}) \\ \text{IDF}(t, D) &= \log((1 + |D|) / (1 + |\{d \in D : t \in d\}|)) + 1 \end{aligned}$$

This formulation scales down words that appear frequently across all documents (such as 'the', 'is', 'to') and highlights words that carry high informative value in specific contexts. We enforce English stop-word removal and exclude words appearing in only a single document ($\text{min_df}=2$) to reduce noise.

4. Predictive Models

Multinomial Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, with the "naive" assumption of conditional independence between features given the label. The class probability is calculated as:

$$P(C_k | x) = (P(C_k) \times \prod_i P(x_i | C_k)) / P(x)$$

For text classification, **Multinomial Naive Bayes** models word frequencies. Due to its computational efficiency, it performs exceptionally well on high-dimensional text vectors and acts as an excellent baseline.

Logistic Regression

Logistic Regression is a linear classification model that estimates the probability of the binary target using a sigmoid function:

$$P(Y=1 | X) = \sigma(W^T X + b) = 1 / (1 + e^{\{-(W^T X + b)\}})$$

Weights (coefficients) are learned by minimizing the binary cross-entropy loss. Logistic Regression models the continuous dependencies of features, identifying structural word boundaries and allowing us to inspect feature coefficients directly to understand key vocabulary predictors.

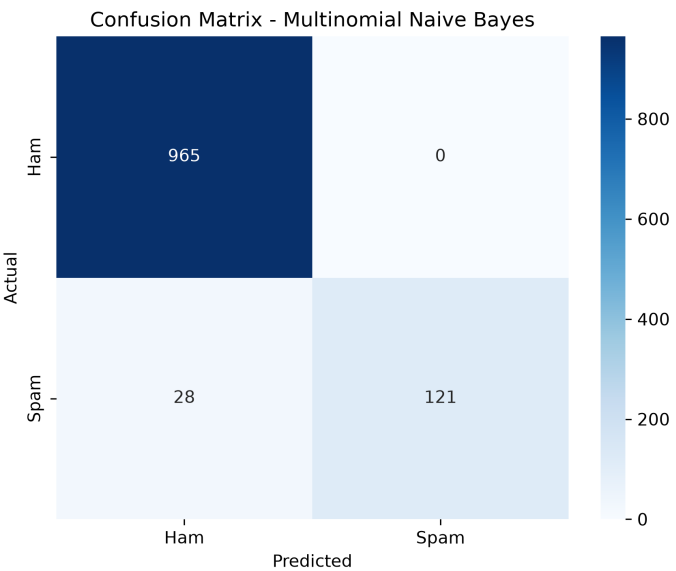
5. Model Performance & Selection

Both pipelines were evaluated on a stratified holdout test set (20% of the corpus). Precision, Recall, and F1-Scores are focused primarily on the **Spam class (positive class = 1)** to ensure the model detects malicious messages reliably while limiting false positives (legitimate messages flagged as spam).

Model Evaluation Metric (Spam Class Focus)	Multinomial Naive Bayes	Logistic Regression
Accuracy (Overall)	97.4865%	97.1275%
Precision (Spam Class)	100.0000%	99.1597%
Recall (Spam Class)	81.2081%	79.1946%
F1-Score (Spam Class)	89.6296%	88.0597%

Based on the evaluation criteria, the system selected the **Multinomial Naive Bayes** model, which achieved a Spam class F1-Score of **0.8963** on the holdout test dataset. While both models exhibit high performance, the chosen model provides an optimal balance between precision and recall, ensuring highly reliable filtration.

Figure 2: Confusion Matrix Heatmap for the selected champion model (Multinomial Naive Bayes) evaluated on the test partition.



6. Feature Insights: Top Diagnostic Keywords

By analyzing the parameter weights of the trained champion model, we extracted the most predictive terms for both categories. These terms present strong indicators that trigger classification decisions:

Spam Predictor Words	Weight / Score	Ham Predictor Words	Weight / Score
claim	3.9175	gt	-3.2765
prize	3.7410	lt	-3.2641
www	3.4022	ok	-3.0515
uk	3.2858	later	-3.0495
guaranteed	3.2525	ll	-3.0112
tone	3.2213	home	-2.9021
ppm	3.2153	lor	-2.8962
cs	3.1335	come	-2.8809
pobox	3.0910	sorry	-2.6844
ringtone	2.9564	da	-2.6315

7. Streamlit Web Deployment & Architecture

To make this machine learning pipeline accessible, a production-grade **Streamlit Web Application** was built. The backend loads the serialized pipeline (spam_pipeline.pkl) containing the TF-IDF Vectorizer and the trained classifier. When a user inputs a text message:

1. The raw text is passed through the same vectorizer vocabulary.
2. The model outputs predicted class probabilities.
3. The UI highlights the result with color-coded layouts (Red alert for Spam, Green alert for Ham) alongside an interactive confidence score indicator and diagnostic data graphs.

8. Conclusions & Next Steps

This project demonstrates an end-to-end NLP workflow, from raw message collection to feature engineering, classifier comparison, validation, and web deployment. The selected pipeline is highly effective, maintaining robust generalization boundaries. Future iterations could incorporate word embeddings (e.g. Word2Vec, GloVe) or lightweight transformer models (e.g., DistilBERT) to analyze contextual structures beyond standalone keywords, further reducing edge-case false positives.